Swift 6 Migration

Divya Maloo

Swift 6 Migration

Swetal Matkar

Swift Concurrency in a nutshell

Swift Concurrency in a nutshell

- Async / Await
- Tasks
- Actors
- Sendable

Async / Await

```
func fetchThumbnail(for id: String, completion: @escaping (UIImage?, Error?) -> Void) {
    let request = thumbnailURLRequest(for: id)
    let task = URLSession.shared.dataTask(with: request) { data, response, error in
        if let error = error {
            completion(nil, error)
        } else if (response as? HTTPURLResponse)?.statusCode != 200 {
            completion(nil, FetchError.badID)
        } else {
            guard let image = UIImage(data: data!) else {
                completion(nil, FetchError.badImage)
                return
            }
            image.prepareThumbnail(of: CGSize(width: 40, height: 40)) { thumbnail in
                guard let thumbnail = thumbnail else {
                    completion(nil, FetchError.badImage)
                    return
                }
                completion(thumbnail, nil)
            }
        }
    }
    task.resume()
}
```

Async / Await

```
func fetchThumbnail(for id: String) async throws -> UIImage {
    let request = thumbnailURLRequest(for: id)
    let (data, response) = try await URLSession.shared.dataTask(with: request)

    guard (response as? HTTPURLResponse)?.statusCode == 200 else { throw FetchError.badID }

    let image = UIImage(data: data)
    guard let thumbnail = await image?.thumbnail else { throw FetchError.badImage }
    return thumbnail
}
```

Task

Flavors of Task

- Async let
- withTaskGroup
- Task
- Task.detach

Flavors of Tasks

- Async let
- withTaskGroup
- Task
- Task.detach

```
func loadUserDataWithoutAsyncLet() async {
    // Both tasks start immediately and run concurrently
     let username = await fetchUsername()
     let network = await fetchNetworks()

    print("Name \(username), Network count \(network.count)")
}
```

Flavors of Tasks

- Async let
- withTaskGroup
- Task
- Task.detach

```
func loadUserData() async {
    // Both tasks start immediately and run concurrently
    async let username = fetchUsername()
    async let network = fetchNetworks()

    print("Name \( await username), Network count  \(await network.count)")
}
```

Flavors of Task

- Async let
- withTaskGroup
- Task
- Task.detach

```swift
func downloadImages(from urls: [URL]) async throws -> [UIImage] {
    // 1. Create a throwing task group.
    // 'of: Data.self' specifies that each child task returns Data.
    // 'returning: [UIImage].self' specifies that the whole group returns an array of UIImages.
    let images = try await withThrowingTaskGroup(of: Data.self, returning: [UIImage].self) { group in

        // 2. Add a dynamic number of tasks to the group.
        for url in urls {
            group.addTask {
                // Perform the asynchronous network request within the child task.
                // This will run concurrently with other tasks in the group.
                let (data, _) = try await URLSession.shared.data(from: url)
                return data
            }
        }

        // 3. Collect the results as they complete.
        var collectedImages: [UIImage] = []
        // The task group acts as an AsyncSequence, yielding results as soon as they are ready.
        for try await data in group {
            if let image = UIImage(data: data) {
                collectedImages.append(image)
            } else {
                // Handle cases where data couldn't be converted to an image
                print("Could not create image from data")
            }
        }

        // 4. Return the final collection once all tasks are finished.
        return collectedImages
    }

    return images
}
```

Flavors of Task

- Async let
- withTaskGroup
- Task
- Task.detach

```
func handleButtonClickAction() {
    Task {
        await fetchUpdates()
    }
}
```

Flavors of Task

- Async let
- withTaskGroup
- Task
- Task.detach

```
func handleButtonClickAction() {
    Task.detached {
        await fetchUpdates()
    }
}
```

Actors

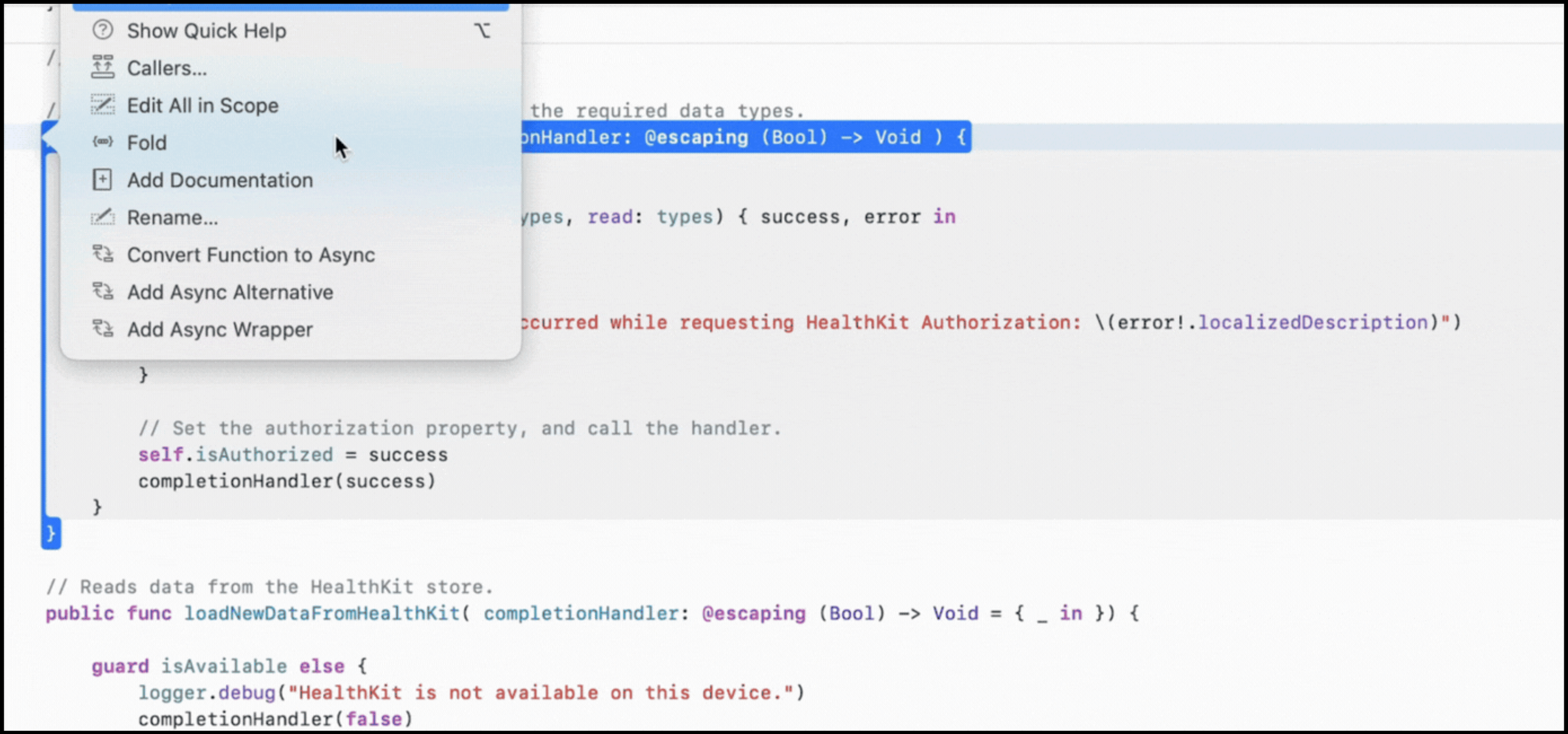
```
actor BankAccount {
  let accountNumber: Int
  var balance: Double

  init(accountNumber: Int, initialDeposit: Double) {
    self.accountNumber = accountNumber
    self.balance = initialDeposit
  }
}
```

Actors

```
extension BankAccount {
  enum BankError: Error {
    case insufficientFunds
  }

  func transfer(amount: Double, to other: BankAccount) throws {
    if amount > balance {
      throw BankError.insufficientFunds
    }

    print("Transferring \(amount) from \(accountNumber) to \(other.accountNumber)")

    balance = balance - amount
    other.balance = other.balance + amount  // error: actor-isolated property 'balance' can only be referenced on 'self'
  }
}
```

Sendable

Quick tip:
for adapting to Swift Concurrency


Show Quick Help ⌥
Callers...
Edit All in Scope
Fold
Add Documentation
Rename...
Convert Function to Async
Add Async Alternative
Add Async Wrapper
the required data types.
onHandler: @escaping (Bool) -> Void) {
ypes, read: types) { success, error in
ccurred while requesting HealthKit Authorization: \(error!.localizedDescription)")
}
// Set the authorization property, and call the handler.
self.isAuthorized = success
completionHandler(success)
}
}
// Reads data from the HealthKit store.
public func loadNewDataFromHealthKit(completionHandler: @escaping (Bool) -> Void = { _ in }) {
guard isAvailable else {
logger.debug("HealthKit is not available on this device.")
completionHandler(false)

Swift 6

When to migrate to Swift 6

- You’re experiencing hard-to-reproduce crashes
- You’re integrating more concurrency into your app
- You’re maintaining a library that is used from concurrent code

Strategy to migrate to Swift 6

Strategy to migrate to Swift 6

The approach has 3 key steps:

1. Select a module

2. Enable swift 6 upcoming concurrency feature with Swift 5

3. Address Warnings

```
.target(
  name: "MyTarget",
  swiftSettings: [
    .enableExperimentalFeature("StrictConcurrency")
  ]
)
```

```
.target(
  name: "MyTarget",
  swiftSettings: [
    .enableUpcomingFeature("StrictConcurrency")
  ]
)
```

```
// In a Settings.xcconfig

SWIFT_STRICT_CONCURRENCY = complete;
```

Enabling Swift 6

```
// swift-tools-version: 6.0

let package = Package(
    name: "MyPackage",
    products: [
        // ...
    ],
    targets: [
        // Uses the default tools language mode (6)
        .target(
            name: "FullyMigrated",
        ),
        // Still requires 5
        .target(
            name: "NotQuiteReadyYet",
            swiftSettings: [
                .swiftLanguageMode(.v5)
            ]
        )
    ]
)
```

Enabling Swift 6

```swift
// swift-tools-version: 6.0

let package = Package(
    name: "MyPackage",
    products: [
        // ...
    ],
    targets: [
        // Uses the default tools language mode (6)
        .target(
            name: "FullyMigrated",
        ),
        // Still requires 5
        .target(
            name: "NotQuiteReadyYet",
            swiftSettings: [
                .swiftLanguageMode(.v5)
            ]
        )
    ]
)
```

package@swift-5.9.swift

```swift
// swift-tools-version: 5.9

let package = Package(
    name: "MyPackage",
    products: [
        // ...
    ],
    targets: [
        .target(
            name: "FullyMigrated",
        ),
        .target(
            name: "NotQuiteReadyYet",
        )
    ]
)
```

Setting XCConfig

You can also set the `SWIFT_VERSION` setting to 6 in an xcconfig file:

```
// In a Settings.xcconfig

SWIFT_VERSION = 6;
```

Strategy to migrate to Swift 6

The approach has 3 key steps:

1. Select a module

2. Enable swift 6 upcoming concurrency feature with Swift 5

3. Address Warnings

Common Compiler Error

Unsafe global variable

```
var supportedStyleCount: Int = 1
```

```
Unsafe Global and Static Variables
  |- error: global variable 'supportedStyleCount' is not concurrency-safe
because it is non-isolated global shared mutable state
  |              |- note: convert 'supportedStyleCount' to a 'let' constant
to make the shared state immutable
  |              |- note: restrict 'supportedStyleCount' to the main actor if
it will only be accessed from the main thread
  |              |- note: unsafely mark 'supportedStyleCount' as concurrency-
safe if all accesses are protected by an external synchronization mechanism
```

Non Sendable Types

```
class WindowStyler {
    var background: ColorComponents
    static let defaultStyler = WindowStyler()

    init(background: ColorComponents = Colo
        self.background = background
    }
}
```

Static property 'defaultStyler' is not concurrency-safe because non-'Sendable' type 'WindowStyler' may have shared mutable state

Class 'WindowStyler' does not conform to the 'Sendable' protocol — Show

Add '@MainActor' to make static property 'defaultStyler' part of global actor 'MainActor' — Apply

Disable concurrency-safety checks if accesses are protected by an external synchronization mechanism — Apply

Generate Fix for Issue — Generate